CTF Writeup

Loaded Dice — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** Loaded Dice **Author:** LordSudo **Difficulty:** Easy **Points:** 100 **Category:** Smart Contract — Predictable Randomness **Result:** Solved (1/1 flag captured)

Description

> A "provably fair" lottery's winner was correctly predicted by a third party before the draw transaction was even mined. You're given the contract source, the entrant list, and the exact block data recovered from node logs. Reconstruct the winner-selection formula offline and predict the winner yourself.

Vulnerability

Weak randomness — winner selection uses inputs that are fully public before the draw transaction executes (`blockhash(block.number - 1)` and `block.timestamp`). Any observer can compute the winning index in advance. (CWE-330: Use of Insufficiently Random Values.)

The contract also carries a decoy: `lastDifficultyObserved = block.prevrandao` is stored but explicitly never used in the selection formula (source comment admits it's leftover from an earlier draft).

Flag

`r00t{1000000000000000000000000000000000000003}`

(Format note: address without the `0x` prefix.)

Winner-Selection Formula (from Lottery.sol)

```
uint256 idx = uint256(
    keccak256(abi.encodePacked(blockhash(block.number - 1), block.timestamp))
) % entrants.length;
winningIndex = idx;
winner = entrants[idx];
```

Both inputs are known before the draw is mined: - `blockhash(block.number - 1)` = the parent block's hash - `block.timestamp` = the draw block's timestamp

Inputs (from block_data.json / entrants.json)

- parentBlockHash: 0xbd14e98d019d0e4471775d86f8c6f2b36eb94c59edd5e2e9e005b503e58e97be

- drawTimestamp: 1735000000
- entrants.length: 8

Reconstruction

Reimplemented the formula in Python (`eth_hash.auto.keccak`):

```
import json
from eth_hash.auto import keccak

entrants = json.load(open("entrants.json"))
block    = json.load(open("block_data.json"))

parent_hash = bytes.fromhex(block["parentBlockHash"][2:])   # 32 bytes
ts_bytes    = block["drawTimestamp"].to_bytes(32, "big")    # uint256

packed = parent_hash + ts_bytes                              # abi.encodePacked
h   = int.from_bytes(keccak(packed), "big")
idx = h % len(entrants)

print(idx, entrants[idx])
```

Output: keccak: c31d1da5b1017a5730ce6f861ca6f9ede8b9a3ba4a3fd4ed5218ba9e4040bce2
index : 2 winner: 0x1000000000000000000000000000000000000003

Flag formatting

Submitted without the `0x` prefix:

```
r00t{1000000000000000000000000000000000000003}
```

Issues Hit & How They Were Solved

1. **Folder names contained spaces**, breaking a naive `for f in $(find ...)` loop (`./Loaded` / `Dice/...` split into separate tokens). Fix: quote paths (`cd "Loaded Dice"`) or use `find -print0 | while IFS= read -r -d ''`.
2. **Decoy variable** — the contract stores `block.prevrandao` in `lastDifficultyObserved`, which tempts you to include it in the formula. The inline comment confirms it's unused. Lesson: read the whole source, don't assume every field participates.
3. **Flag encoding had no `0x` prefix** — assumed format with prefix failed. Retried without, accepted.

Key Takeaways

- On-chain "randomness" drawn from block fields is predictable; `blockhash`, `block.timestamp`, `block.prevrandao`, and `block.difficulty` are all visible to miners / observers before inclusion.
- "Provably fair" only holds if the seed is committed to *after* the outcome is fixed and cannot be ground by the block producer — a commit-reveal scheme, VRF (e.g. Chainlink VRF), or an off-chain oracle is required.
- Read the entire contract; decoy fields are a standard CTF misdirection.

Tools Used

python3, eth-hash (pycryptodome backend), apktool not needed (source provided)